pdf4tcl

Creating PDFs, forms, CID fonts, encryption

2026-10-26

Contents

1 Understanding pdf4tcl **6**
1.1 Introduction 6
1.2 First Document 6
1.3 What pdf4tcl Is — and What It Is Not 6
1.4 Coordinates and Units 6
1.5 Warnings 7
1.6 Related Sources 7
1.7 Summary 7
2 Document and Page **7**
2.1 Introduction 7
2.2 Constructor 7
2.3 Page 8
2.4 Writing 8
2.5 Margin and Origin 9
2.6 Summary 9
3 Text and Standard Fonts **9**
3.1 Introduction 9
3.2 Setting a Font and Writing 9
3.3 ToUnicode 9
3.4 Width and Box 9
3.5 PDF/A and Standard Fonts 10
3.6 Summary 10
4 CID and Unicode **10**
4.1 Introduction 10
4.2 Recipe 10
4.3 Two Paths, Two Ceilings 11
4.4 Checking Substitutions 11
4.5 Tcl 8.6 and Characters Above U+FFFF 12
4.6 Encryption 12
4.7 Summary 12
5 Graphics **12**
5.1 Introduction 12
5.2 Color 12
5.3 Shapes 13
5.4 Transparency and Gradients 13
5.5 Transformation 13
5.6 Summary 14
6 Images **14**
6.1 Introduction 14
6.2 PNG, JPEG, TIFF 14
6.3 Reuse 14
6.4 Tagged and PDF/UA 14
6.5 Summary 15
7 Canvas Export **15**
7.1 Introduction 15

7.2 Classic Canvas 15
7.3 tkpath and tko::path 16
7.4 Font Mapping 16
7.5 Text Baseline 16
7.6 Summary 16
8 Creating Forms 17
8.1 Introduction 17
8.2 Text Field and Buttons 17
8.3 Options 17
8.4 Sum (JS in the Viewer) 18
8.5 PDF/A and Check Marks 18
8.6 Layers and Fields 18
8.7 Summary 18
9 fillForms 19
9.1 Introduction 19
9.2 Reading, Filling, Exporting 19
9.3 What getForms Returns 19
9.4 Appearance Since 0.9.4.64 20
9.5 NeedAppearances Only with an Old Stream 20
9.6 Values 20
9.7 Field Tree (0.9.4.64) 20
9.8 Round Trip 21
9.9 Dynamic XFA 21
9.10 Summary 21
10 Annotations 21
10.1 Introduction 21
10.2 Common Calls 21
10.3 What Draws Itself 22
10.4 A Stamp Needs a Font 22
10.5 Arrowheads 22
10.6 Links 22
10.7 Tagged 23
10.8 Summary 23
11 Encryption 23
11.1 Introduction 23
11.2 AES-128 24
11.3 AES-256 24
11.4 CID Fonts: WriteObjectDirect (0.9.4.67) 24
11.5 Checking 25
11.6 Summary 25
12 catPdf 25
12.1 Introduction 25
12.2 Recipe 25
12.3 Mixing Tagged Files 25
12.4 Forms (0.9.4.44) 26
12.5 XRef Streams 26
12.6 Limits 26

12.7 Summary 26
13 PDF/A 27
13.1 Introduction 27
13.2 Level B 27
13.3 Level U (since 0.9.4.56) 27
13.4 Level A (since 0.9.4.41) 27
13.5 Pitfalls 28
13.6 Summary 28
14 Layers 28
14.1 Introduction 28
14.2 Recipe 28
14.3 Options (since 0.9.4.63) 29
14.4 Counting 30
14.5 Forms 30
14.6 Summary 30
15 Metadata 30
15.1 Introduction 30
15.2 Setting 30
15.3 Viewer 30
15.4 Attachments 31
15.5 Encryption 31
15.6 catPdf 31
15.7 Summary 31
16 gsave and Text 31
16.1 Introduction 31
16.2 The Trap Up to 0.9.4.61 31
16.3 Since 0.9.4.62 32
16.4 Other Methods 32
16.5 Rule 32
16.6 Tagged Text Since 0.9.4.69 32
16.7 Summary 33
17 CID Subset and CMap 33
17.1 Introduction 33
17.2 cmap Is Required 33
17.3 Identity, Not Unicode 33
17.4 letzteGlyphe Is numGlyphs-1 34
17.5 Checksums (0.9.4.68) 34
17.6 CFF 34
17.7 Summary 34
18 Case Study: Report 34
18.1 Introduction 34
18.2 Font and Document 34
18.3 Header and Key Figures 35
18.4 Field and Stamp 35
18.5 Filling Later 36
18.6 What Is Deliberately Missing 36
18.7 Summary 36

19 Pitfalls **36**
19.1 Text and Fonts 36
19.2 Graphics 37
19.3 Canvas 37
19.4 Forms and Annotations 37
19.5 Output 37
20 API **37**
20.1 Package and Object 38
20.2 Text and Fonts 38
20.3 Graphics 38
20.4 Images and Canvas 39
20.5 Form, Annotation, Layer 39
20.6 Concatenation, Metadata, PDF/A 39
21 Glossary **39**
21.1 A 39
21.2 B 40
21.3 C 40
21.4 N 40
21.5 O 40
21.6 P 40
21.7 T 40
21.8 W 40
21.9 X 41
22 Error Messages **41**
22.1 no font set 41
22.2 unknown color: "1" -- expected a name like "red", "#rrggbb", ... 41
22.3 unknown color / invalid ... component 41
22.4 createFontSpecCID: base font '...' not loaded 41
22.5 invalid TTF file checksum 41
22.6 fillForms: no such field(s) 41
22.7 fillForms: "..." is not an option of "..." 41
22.8 unknown form type ... 41
22.9 unknown metadata option "..." 41
22.10 papersize "..." is unknown 42
22.11 rotation ... not a multiple of 90 42
22.12 endLayer without beginLayer / layer still open 42
22.13 Adobe: Blank Page, No Message 42
22.14 qpdf: inflate: incorrect header check 42
22.15 PDF/A: the standard font Helvetica has no embeddable font program 42
22.16 pdf4tcl: linkAdd target(s) never defined with destinationAdd: ... 42
22.17 destination "..." already defined 42
22.18 invalid destination name "...": must match [A-Za-z0-9._-]+ 42
23 License **42**

1 Understanding pdf4tcl

1.1 Introduction

pdf4tcl writes PDF from Tcl. One class, methods on the document object, output through `write`. This chapter establishes what the package is good for and where other layers take over.

The book describes **0.9.4.69**: tagged text stays inside BT/ET. On top of that come the links within the document from **0.9.4.72** (chapter Annotations). CID subset with identity CMap and table checksums since 0.9.4.68, encrypted CID fonts through `WriteObjectDirect` since 0.9.4.67, appearance streams for stamps and `fillForms`.

1.2 First Document

```
package require pdf4tcl 0.9.4.69

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient true -compress 1]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Hallo Welt" -x 50 -y 50
$pdf endPage
$pdf write -file ausgabe.pdf
$pdf destroy
```

%AUTO% assigns the object name. `-orient true` (default) puts Y at the top, as in Tk. `-compress 1` (default) compresses content streams.

Without `startPage`, the first drawing operation starts the page itself. Still set `startPage` / endPage explicitly: margin, paper and open layers then stay easy to follow.

1.3 What pdf4tcl Is — and What It Is Not

Suitable:

- Pages with text, vectors, images
- Unicode through a CID font
- Creating AcroForms and filling them later
- Canvas export, AES encryption, PDF/A, layers
- Appending several PDFs to one another (`catPdf`)

Not suitable as the only layer:

- Tables with page breaks, running text across columns, label sheets
- `pdf4tcllib` provides that on top of the same drawing API

pdf4tcl draws. Layout remains the job of the caller or of the companion library.

1.4 Coordinates and Units

The default unit is the point (`-unit p`). Millimeters: `-unit mm`. With `-orient 1` the origin lies at the top left of the type area (after `-margin`). Internally PDF counts Y from the bottom; the class converts.

Colors range over **0.0.. 1.0**, not 0..255. Since 0.9.4.39 a value outside that range raises an error instead of being written silently.

1.5 Warnings

Some defects do not interrupt: a standard font under `-pdfa`, alpha under PDF/A-1. Check the list after `write`:

```
if {[llength $::pdf4tcl::warnings]} {
    puts $::pdf4tcl::warnings
}
```

1.6 Related Sources

- Manpage `pdf4tcl.md` — every method
- `pdf4tcl/doc/en/howtos/` — one task per file, with a script
- `pdf4tcl/UPGRADING.md` — what changes for existing code
- `pdf4tcl/demo/` — one feature per program

This book presents the material as a textbook. The howtos remain the runnable test.

1.7 Summary

- `pdf4tcl::new`, page, draw, `write`, `destroy`.
- Version **0.9.4.69** for tagged text in BT/ET, CID cmap, AES with embedded font and appearance streams.
- Layout helpers live outside this package.

Next chapter: Document and Page

2 Document and Page

2.1 Introduction

A document is created in the constructor. Each page starts with `startPage` and ends with endPage. Options on the constructor act as defaults; `startPage` may override paper, margin and orientation.

2.2 Constructor

```
set pdf [::pdf4tcl::new %AUTO% \
        -paper a4 \
        -orient 1 \
        -margin 50 \
        -compress 1]
```

Common options:

Option	Default	Meaning
-paper	a4	format name or {breite höhe} (width height)
-orient	1	Y from the top
-landscape	0	swap width and height
-margin	0	one, two or four values
-unit	p	p, mm, cm, i
-compress	1	compress content streams
-cmyk	0	DeviceCMYK instead of RGB
-pdfa	""	conformance level, see PDF/A
-userpassword	""	see Encryption

Paper sizes: the A series, since 0.9.4.25 also the B and C series as well as 4A0 and 2A0. The A3 height is **1191.0** pt (previously 1190.0). Tests that hard-code the old number should use `getPaperSize`.

```
lassign [pdf4tcl::getPaperSize a3] w h
lassign [$pdf getPageSize] pw ph
```

`getPageSize` returns the current page in the document unit.

2.3 Page

```
$pdf startPage
# zeichnen
$pdf endPage
$pdf startPage -paper a5 -margin 20
# zweite Seite kleiner
$pdf endPage
```

`startPage` on an open page calls `endPage` itself. An optional content group (layer) that is still open (`beginLayer` without `endLayer`) makes `endPage` fail.

2.4 Writing

```
$pdf write -file bericht.pdf
$pdf write -chan $ch
$pdf write
```

`-file` opens and closes the file. `-chan` writes to an open channel (since 0.9.4.25). Without an option the output goes to stdout. Then `destroy`.

`finish` closes the document; `write` calls it internally. Do not continue drawing after `finish`.

2.5 Margin and Origin

`-margin 50` sets all four sides. Two values: left/right, top/bottom. Four values: left, right, top, bottom — in the document unit.

With `-orient 1`, `-x 0 -y 0` is the top left corner of the type area, not of the paper. Without a margin the two coincide.

2.6 Summary

- The constructor sets defaults; `startPage` may deviate.
- `-orient 1`: Y at the top. Default unit is the point.
- `write -file` or `write -chan`, then `destroy`.

Next chapter: Text and Standard Fonts

3 Text and Standard Fonts

3.1 Introduction

Fourteen standard fonts (Base-14) never need to be embedded: Helvetica, Times, Courier, each in four styles, plus Symbol and ZapfDingbats. The character repertoire is WinAnsi / CP1252. Everything outside it becomes?.

3.2 Setting a Font and Writing

```
$pdf setFont 18 Helvetica-Bold
$pdf text "Titel" -x 0 -y 24
$pdf setFont 12 Times-Roman
$pdf text "Fließtext mit Umlauten: äöü ÄÖÜ ß" -x 0 -y 50
$pdf setFont 10 Courier
$pdf text "mono 0123456789" -x 0 -y 70
```

Name the styles exactly: Helvetica **-Oblique**, Times **-Italic**. `Helvetica-Italic` does not exist.

`text` deliberately leaves the text object (BT … ET) open, so that several calls share one block. Other drawing methods close it. Details: gsave and Text.

3.3 ToUnicode

Since 0.9.4.9 standard fonts carry a ToUnicode CMap for WinAnsi. Copying and `pdftotext` return Latin umlauts correctly. Greek and Cyrillic remain? — for those see CID and Unicode.

`getSubstCount` counts substitutions — **under Tcl 9**. Under 8.6 `encoding convertto` substitutes silently and the counter stays 0. The page looks the same in both generations.

3.4 Width and Box

```
set w [$pdf getStringWidth "Rechnung 2026"]
set w [$pdf getStringWidth "Probe" -font Helvetica -size 12]
```

Without a preceding `setFont`, use the variant with `-font` / `-size` (since 0.9.4.23).

```
$pdf drawTextBox 0 80 400 120 $absatz -align left -newyvar nextY
$pdf line 0 $nextY 400 $nextY
```

`-newyvar` returns Y in the caller's coordinates. Before 0.9.4.44, with `-orient 1` it returned an internal page coordinate; code that computed `pageHeight - $nextY` back then must remove that correction.

Soft hyphen `U+00AD`: since 0.9.4.61 a break suggestion, not a glyph. `text`, `drawTextBox` and `getStringWidth` treat it the same way. `Sil\u00ADben` is as wide as `Silben`.

3.5 PDF/A and Standard Fonts

Base-14 has no embeddable font program. PDF/A and PDF/UA require one. Since 0.9.4.41 this is reported in `::pdf4tcl::warnings`; the file is still created and fails veraPDF. For archiving: load TrueType, see the next chapter.

3.6 Summary

- Base-14 without embedding, Latin-1 only.
- Styles: `-Oblique` for Helvetica, `-Italic` for Times.
- `drawTextBox` `-newyvar` in caller coordinates (0.9.4.44).
- Soft hyphen invisible and without width since 0.9.4.61.

Next chapter: CID and Unicode

4 CID and Unicode

4.1 Introduction

`createFont` with WinAnsi covers Latin-1. Polish, Greek, Cyrillic, mathematics or CJK need a CID font: load TrueType, create a CID specification, select it with `setFont`.

4.2 Recipe

```
pdf4tcl::loadBaseTrueTypeFont BaseDejaVu \
        /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
pdf4tcl::createFontSpecCID BaseDejaVu Uni

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf setFont 12 Uni
$pdf text "Grüße aus München, Polska, Ellada" -x 0 -y 24
$pdf endPage
$pdf write -file unicode.pdf
$pdf destroy
```

Three steps: load the base, bind the CID name, use that name in `setFont`. Without a loaded base:

```
createFontSpecCID: base font '…' not loaded (use loadBaseTrueTypeFont
    first)
```

The loader rejects color bitmap emoji (CBDT/CBLC) and COLR. Outline fonts only.

4.3 Two Paths, Two Ceilings

Path	Call	Ceiling
encoded subset	createFont / createFontSpecEnc	at most 256 code points
CID	createFontSpecCID	the repertoire actually used, Identity-H

Since 0.9.4.57 **both paths subset**. Before that, CID put the whole file into the PDF. File size for one line of Greek in DejaVu Sans:

Variant	File
subset, 5 code points	8 860 bytes
subset, 200 code points	29 761 bytes
CID after subsetting	50 004 bytes
CID before 0.9.4.57 (whole face)	386 728 bytes

CFF/OpenType stays complete; pdf4tcl subsets TrueType. In that case a warning appears in `::pdf4tcl::warnings`.

With an encoded subset, include? (63) in the list. Characters outside it otherwise fall back to slot 0 — the first code point given. A subset that starts with `G` turns Greek into `GGGGGG`.

4.4 Checking Substitutions

```
$pdf text $zeile -x 0 -y 40
puts [$pdf getSubstCount]
```

`0` means: every character had a glyph. Question marks on the page without an error: the face lacks the code point, or Base-14 was in use.

4.5 Tcl 8.6 and Characters Above U+FFFF

As a literal in the source:

	string length	first character
Tcl 9	1	U+1F600
Tcl 8.6	1	U+FFFD

Read from a UTF-8 file, both generations get it right. No `\U` escapes above U+FFFF in code that must run under 8.6. Measure width with `getStringWidth`, not with `string length`.

4.6 Encryption

Since **0.9.4.67**, embedded CID fonts in an AES document take the same write path as other objects (`WriteObjectDirect`). Regenerate files from 0.9.4.66. See Encryption.

Since **0.9.4.68** the subset file itself carries a valid cmap and table checksums. In depth: CID Subset and CMap.

4.7 Summary

- `loadBaseTrueTypeFont`, then `createFontSpecCID`, then `setFont`.
- CID subsets TrueType since 0.9.4.57; CFF stays whole.
- Encoded subset: 256 code points,? in the list.
- `getSubstCount` after drawing.

Next chapter: Graphics

5 Graphics

5.1 Introduction

Lines, areas, color, transparency and gradients are handled by methods on the document. Since 0.9.4.39 color shares one pipeline: fill, stroke, gradient, form, annotation.

5.2 Color

```
$pdf setFillColor 1 0 0
$pdf setFillColor #cc3300
$pdf setFillColor red
$pdf setFillColor 0 0.5 1 0
$pdf setStrokeColor navy
$pdf setBgColor 1 1 0
```

Components **0.0.. 1.0**. `255 0 0` has been an error since 0.9.4.39 (before that the file

contained the illegal operand and viewers clamped silently). 147 X11/CSS names without Tk. Names outside the table are looked up with `winfo rgb` when Tk is present.

`setBgColor` applies only to `text -bg`, not to the page. Page background: draw a full-page rectangle first.

5.3 Shapes

```
$pdf setStrokeColor 0 0 0
$pdf setLineWidth 1
$pdf line 0 36 250 36

$pdf setFillColor 0.9 0.9 0.9
$pdf rectangle 0 50 250 60 -filled 1
```

Further methods: `circle`, `oval`, `polygon`, `arc`, `roundedRect`, `arrow`. After colored text, set the fill color back to black, otherwise the next box inherits red.

5.4 Transparency and Gradients

```
$pdf setAlpha 0.5
$pdf rectangle 40 40 120 80 -filled 1
$pdf setAlpha 1
```

Under `-pdfa 1b` / `1a`, alpha below 1 violates PDF/A-1. Since 0.9.4.22 a warning appears in `::pdf4tcl::warnings`; the file is still created.

```
$pdf gsave
$pdf clip 50 100 400 60
$pdf linearGradient 50 130 450 130 red blue
$pdf grestore
$pdf rectangle 50 100 400 60
```

Radial: inner center/radius, outer center/radius, two colors. `-extend {0 0}` stops at the ends. With `-cmyk 1` the shading writes `/DeviceCMYK`.

```
$pdf setBlendMode Multiply
# überlappende Füllungen
$pdf setBlendMode Normal
```

5.5 Transformation

```
$pdf gsave
$pdf translate 200 300
$pdf rotate 45
$pdf line 0 0 80 0
$pdf grestore
```

`rotate`, `scale`, `translate` (since 0.9.4.20) set `cm` and record `rawcoords`. Always pair

them with `gsave` / `grestore`.

Text does not follow cm. `text` sets `Tm` absolutely. Rotated text: still use `translate` / `rotate`, then `text` at `-x 0 -y 0` — and read gsave and Text first: `gsave` closes an open `BT`.

5.6 Summary

- Color 0..1, names without Tk, range check since 0.9.4.39.
- Clip the gradient area, then `linearGradient` / `radialGradient`.
- Transformations always between `gsave` and `grestore`.

Next chapter: Images

6 Images

6.1 Introduction

A raster image gets onto the page in two steps: `addImage` takes the file into the document, `putImage` places it. The same ID may appear several times.

6.2 PNG, JPEG, TIFF

```
set id [$pdf addImage /pfad/foto.png]
$pdf addImage $pfad -id logo
$pdf putImage $id 50 100 -width 200
```

Without `-height` the height follows the aspect ratio. `-id` assigns a fixed name instead of a number.

JPEG and many PNG/TIFF variants work. Since **0.9.4.28** Adam7-interlaced PNG is de-interlaced on embedding. The loader rejects interlaced PNG below 8 bits with a clear message.

Unsupported format: load it through Tk/Img, then use `addRawImage` / `putRawImage`.

6.3 Reuse

```
set logo [$pdf addImage logo.png -id logo]
$pdf putImage logo 20 20 -width 80
$pdf startPage
$pdf putImage logo 20 20 -width 80
```

The bytes are stored once in the PDF. Each placement is a `Do` on the same XObject.

6.4 Tagged and PDF/UA

Figures need a structure element with `-alt`:

```
$pdf tagBegin Figure -alt "Firmenlogo"
$pdf putImage $id 50 100 -width 120
$pdf tagEnd
```

A form XObject with tagged content may be drawn only **once** (0.9.4.46). Under PDF/UA, veraPDF rule 7.20-2 forbids every further placement; pdf4tcl then warns but does not abort. Reuse remains good PDF — the conformance claim rules it out.

6.5 Summary

- `addImage`, then `putImage` with width or height.
- Interlaced PNG since 0.9.4.28; sub-8-bit interlaced remains an error.
- PDF/UA: `Figure` with `-alt`; do not place an XObject more than once when UA is claimed.

Next chapter: Canvas Export

7 Canvas Export

7.1 Introduction

The canvas method on the document draws a Tk widget onto the page. Call `update` first so that layout and items are in place. Three widget classes, three code paths.

Requires **wish** or a set `DISPLAY`.

7.2 Classic Canvas

```
package require Tk
package require pdf4tcl 0.9.4.69

wm withdraw .
canvas .c -width 400 -height 300 -background white
pack .c
.c create rectangle 10 10 120 80 -fill #cde -outline black
.c create text 65 45 -text "Probe" -anchor center
update idletasks

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set bb [.c bbox all]
$pdf canvas .c -bbox $bb -x 50 -y 50 -width 400 -height 300
$pdf endPage
$pdf write -file canvas.pdf
$pdf destroy
```

`-bbox` is the source area in the canvas. `-x` / `-y` / `-width` / `-height` refer to the PDF page. `-sticky` controls the fit (nw by default). Further options: `-bg`, `-fontmap`, `-textscale`.

Without `update`, `bbox` is often empty or stale. With an empty bbox the method returns silently

and the page stays blank.

7.3 tkpath and tko::path

The same method: `$pdf canvas .path …`. For PathCanvas and TkoPath, `-bbox` is **mandatory**.

The matrix form differs:

Widget	Matrix
tkpath	nested
tko::path	flat {a b c d tx ty}

Do not mix the forms. For tko, pdf4tcl expects the flat form.

window items are often skipped. Register a photo with `addImage` beforehand if the export would otherwise not find the image.

7.4 Font Mapping

Text outside Latin-1: load a TrueType font under the **Tk family name** (since 0.9.4.60); then no `-fontmap` is needed:

```
pdf4tcl::loadBaseTrueTypeFont Base tahoma.ttf
pdf4tcl::createFontSpecCID Base Tahoma
.c create text 20 30 -text "Grüße" -font {Tahoma 14}
$pdf canvas .c -bbox [.c bbox all]
puts [$pdf getSubstCount]
```

Where the name cannot be chosen freely, set `-fontmap`. The mapping always wins over the family lookup.

The key is not the same:

- `tk::canvas`: the **whole** font specification — `{Helvetica 14}` is not found under `Helvetica`. A named font is the safer match.
- `tko::path`: only the **family**.

Wrong key: question marks, no error. Hence `getSubstCount`.

7.5 Text Baseline

PDF text sits on the **baseline**, canvas text at its `-anchor`. A visible offset in Y is expected when nw anchors and uncorrected PDF text are mixed. Inside `$pdf canvas`, pdf4tcl takes over the item anchors.

7.6 Summary

- `update`, then `bbox`, then `$pdf canvas`.
- Three classes: `canvas`, PathCanvas, TkoPath.
- tko: flat matrix. tkpath: nested.

- `-fontmap` or family name; check `getSubstCount`.

Next chapter: Creating Forms

8 Creating Forms

8.1 Introduction

`addForm` places AcroForm fields on the current page. The appearance stream is created along with the field. A document built with its values in a single pass does not need `fillForms`.

Field text is in practice Base-14 / Latin-1. CID in AcroForm is hard; put umlauts in labels on the page as ordinary `text`.

8.2 Text Field and Buttons

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Name:" -x 0 -y 40
$pdf addForm text 60 28 200 16 -id f_name

$pdf addForm pushbutton 0 70 90 20 -id f_reset \
        -caption "Reset" -action reset
$pdf addForm pushbutton 100 70 90 20 -id f_go \
        -caption "Submit" -action submit \
        -url "mailto:bestellung@example.com"
$pdf endPage
$pdf write -file formular.pdf
$pdf destroy
```

Types: `text`, `password`, `checkbutton` (alias `checkbox`), `radiobutton`, `combobox`, `listbox`, `pushbutton`, `signature`.

`-id` takes word characters only, no dot. Nested names occur in the field tree of foreign files, not when creating fields here.

8.3 Options

Option	Effect
-init	initial value, visible everywhere
-align	left / center / right
-multiline	multiple lines
-maxlen / -comb	comb field, cells = width / MaxLen
-options	choice list as labels (no export/label pair)

Option	Effect
-tooltip	tooltip
-readonly / -required	flags
-color -bgcolor -bordercolor -borderwidth	appearance
-layer	/OC on the annotation
-format date	formatting in the viewer
-calculate	sum over field IDs, needs JS in the viewer

`-comb 1` requires `-maxlen`, excludes `-multiline` and honors `-align`. Since 0.9.4.63.

`text`, `password`, `checkbutton` and `radiobutton` work before `setFont`. `combobox`, `listbox`, `pushbutton` and `signature` require a font to be set (message since 0.9.4.63: `no font set`).

Since 0.9.4.63 an empty text field carries an appearance stream (length 0), so that `fillForms` has something to overwrite.

Since 0.9.4.66 an empty field with `-bordercolor` and `-borderwidth` draws the border instead of an empty stream. Without a border the stream stays empty.

8.4 Sum (JS in the Viewer)

```
$pdf addForm text 400 200 90 16 -id b1 -align right -init 120
$pdf addForm text 400 220 90 16 -id b2 -align right -init 80
$pdf addForm text 400 250 90 16 -id f_sum -align right \
        -calculate {sum {b1 b2}} -init 200 \
        -borderwidth 1 -bgcolor {0.95 0.95 0.85}
```

`-init` shows the value everywhere. `-calculate` updates in Acrobat, Firefox, Chromium and Foxit.

8.5 PDF/A and Check Marks

A check mark as a ZapfDingbats glyph cannot be embedded. Since 0.9.4.55 pdf4tcl draws the check mark as a vector as soon as `-pdfa` or PDF/UA is set. Without a conformance claim the glyph remains. `-markstyle font` / `vector` / `auto` (default `auto`).

8.6 Layers and Fields

`beginLayer` does not reach annotations — they lie outside the content stream. `-layer` on `addForm` writes `/OC` into the annotation dictionary.

8.7 Summary

- `addForm TYP x y b h -id name` (type, x, y, width, height).
- The appearance is created along with the field.

- Comb needs `-maxlen`. Whether a font is required depends on the type.
- PDF/A: check mark as a vector (`-markstyle auto`).

Next chapter: fillForms

9 fillForms

9.1 Introduction

Three calls work on a **finished file**, not on the document object: `getForms`, fillForms, `exportForms` (since 0.9.4.50).

9.2 Reading, Filling, Exporting

```
set fields [pdf4tcl::getForms "auftrag.pdf"]
pdf4tcl::fillForms "auftrag.pdf" "ausgefuellt.pdf" {f_name "Meier &
    Co"}
pdf4tcl::exportForms "ausgefuellt.pdf" "ausgefuellt.fdf"
```

`fillForms` writes values and returns how many fields it hit. An unknown name is an error:

```
fillForms: no such field(s) in "auftrag.pdf": kein_feld
```

Fields not named keep what they had.

Encrypted files: `getForms` returns an empty result, `fillForms` finds no fields — there is no `-password`. Build the file unencrypted before filling, or leave out the passwords.

9.3 What getForms Returns

A dict mapping field ID to a dict:

Key	Meaning
type	/Tx, /Btn, /Ch, /Sig
value	current value, unescaped
flags	field flags
default	appearance state for buttons
maxlen	maximum length or empty (0.9.4.63)
comb	1 for a comb field (0.9.4.63)
widgets	number of widgets (0.9.4.64)
description	tooltip (0.9.4.65)
options	allowed values of a choice field (0.9.4.65)

Code that compares the whole dict with a literal will notice the new keys.

9.4 Appearance Since 0.9.4.64

`fillForms` writes the value **and** rebuilds the stream — for a simple text field, a multiline field and a comb **with** `/MaxLen`. One stream per widget, built from that widget's own rectangle.

Up to 0.9.4.63 the stream stayed as it was: a viewer honoring the flag showed the new value, the print path the **old** one. Empty field: nothing on paper.

Deliberately not rebuilt:

Field	Stream
Comb without /MaxLen	old — no divisor, no cells
Password	old — otherwise plain text in the stream
Choice (choice)	old — box and border would be lost
Check / radio	switching between the existing states

9.5 NeedAppearances Only with an Old Stream

fillForms sets NeedAppearances **only** where the stream stays old (since 0.9.4.66). The flag tells the viewer: rebuild everything.

Observed: filling a text field made an **untouched** check box lose its border. Under the flag, PDFium rebuilds every Off appearance and produces an empty one.

Where the stream was drawn by pdf4tcl itself, the flag is left out. Where comb without MaxLen, password or choice keep the old stream, it is set — otherwise the new value would not be visible anywhere.

Code that relied on the flag always being present: fill a field of the old kind, or set the flag yourself.

9.6 Values

Text: a string. Check / radio: state name **with a slash**, as in the file:

```
pdf4tcl::fillForms in.pdf out.pdf {agreed /Yes}
```

Choice: export value or label. Anything else is an error (since 0.9.4.65). An empty value is allowed: nothing selected.

```
fillForms: "Frankreich" is not an option of "land";
allowed: Deutschland Niederlande Belgien
```

Up to 0.9.4.64 every value was written and success was reported; the field then carried a state that no viewer displays.

9.7 Field Tree (0.9.4.64)

`getForms` and `fillForms` walk the tree, not each widget on its own. A field on several sheets (waybill) is found. Nested name: `person.city`, not `city`. Code that used to fill the short name no longer hits the field.

9.8 Round Trip

`getForms` returns text **unescaped**, exactly as `fillForms` takes it (since 0.9.4.55):

```
set fields [pdf4tcl::getForms "in.pdf"]
set values {}
dict for {id info} $fields {
    dict set values $id [dict get $info value]
}
dict set values f_name "Meier & Co (GmbH)"
pdf4tcl::fillForms "in.pdf" "out.pdf" $values
```

Name values (`/Yes`, `/Off`) stay as fillForms expects them for buttons.

9.9 Dynamic XFA

fillForms rejects dynamic XFA forms (0.9.4.64): the pages are generated from XML, and `/V` changes nothing visible. Hybrid XFA (XFA plus usable AcroForm fields) is filled as before.

9.10 Summary

- Appearance for a simple text field rebuilt since 0.9.4.64.
- Comb without MaxLen, password, choice: old stream.
- NeedAppearances only where the stream stays old.
- Names: full chain, `person.city`.

Next chapter: Annotations

10 Annotations

10.1 Introduction

Notes, stamps, markup and lines lie **next to** the content stream. A name is not a drawing. Whether a viewer shows anything depends on the type and on the appearance stream.

10.2 Common Calls

```
$pdf addAnnotNote 100 100 20 20 -content "Prüfen" -author "Lektor" \
        -icon Comment -color {0.6 0.8 1.0}
$pdf addAnnotFreeText 50 200 200 40 "Immer sichtbar" \
        -color {0 0 0} -bgcolor {1 1 0.8}
$pdf addAnnotStamp 300 500 80 30 -name Approved -color {1 0 0}
$pdf addAnnotHighlight 50 300 200 14 -color {1 1 0}
$pdf addAnnotUnderline 50 320 200 14
$pdf addAnnotStrikeOut 50 340 200 14 -color {1 0 0}
$pdf addAnnotLine 50 400 200 400 -color {0 0 0}
```

10.3 What Draws Itself

Kind	Drawn from
Highlight, Underline, StrikeOut	area and color
FreeText	text and colors
Note	the viewer's icon set
Stamp	appearance stream only
Line	appearance stream only

A stamp names which stamp was meant. Acrobat has artwork for the named stamps. **PDFium has none**, and PDFium is the viewer in Chrome and Edge. The same holds for the line: end points say where, not how.

Since **0.9.4.64** `addAnnotStamp` and `addAnnotLine` write an appearance stream. Checked on `demo/demo-annotations.pdf`: none of six stamps and five lines was visible in PDFium; all eleven are now. The file gets slightly larger.

10.4 A Stamp Needs a Font

The appearance of a stamp names a font. Therefore the stream is written **only after setFont**. Without a font set, the annotation is created as before 0.9.4.64 — the call does not fail, PDFium shows nothing.

Order:

```
$pdf setFont 10 Helvetica
$pdf addAnnotStamp 300 500 80 30 -name Approved -color {1 0 0}
```

10.5 Arrowheads

Drawn for `OpenArrow`, `ClosedArrow`, `ROpenArrow`, `RClosedArrow`, `Circle`, `Square`. Every other line ending stays a plain line. A line without a head is half an answer, a wrongly drawn head a wrong one.

10.6 Links

A link is an annotation of type `/Link`: a rectangle that triggers something when clicked. A border is drawn only on request.

hyperlinkAdd opens an address (`/S /URI`):

```
$pdf hyperlinkAdd 50 100 200 20 "https://www.tcl-lang.org"
```

Jumps **within the document** exist since **0.9.4.72**. Two methods belong together. destinationAdd creates a named destination on the current page; linkAdd places a rectangle that jumps to that name:

```
$pdf startPage
$pdf text "1 Einleitung ........ 3" -x 50 -y 100
$pdf linkAdd 50 88 200 14 kap1          ;# Ziel gibt es noch nicht
$pdf endPage
...
$pdf startPage
$pdf text "1 Einleitung" -x 50 -y 60
$pdf destinationAdd kap1 -y 45          ;# Ziel, etwas über der Zeile
```

The link carries only the name (`/Dest /kap1`). It is resolved only when the file is written, through the `/Dests` dictionary in the catalog. Therefore the destination may be created **after** the link — exactly what a table of contents placed before its chapters needs.

- Name: letters, digits, `.` `_` `-`; once per document.
- `-y` (and `-x`) in the current unit; without `-y`, the top edge of the page.
- Border and highlighting as with `hyperlinkAdd`: `-borderwidth -bordercolor -borderradius -borderdash -highlight`.
- A link to a name that no `destinationAdd` creates does not abort anything. The file is written, the link has no effect, and stderr reports the name.
- Allowed under `-pdfa 2b`.

Checked with `tests/linkdest.test` (12 cases) and on docir books: the table of contents of the macOS book, 238 entries, each jumping to the printed page and there to the heading.

10.7 Tagged

Annotations belong to the structure tree only inside an open `tagBegin Link` or `tagBegin Annot` … `tagEnd`. Otherwise they are clickable but invisible to assistive technology. Since 0.9.4.39 this appears once per document in `::pdf4tcl::warnings`. For PDF/UA, wrap the annotation.

10.8 Summary

- Stamp and line need an appearance (0.9.4.64), otherwise PDFium shows nothing.
- `setFont` before `addAnnotStamp`.
- Highlight draws itself from area and color.
- `hyperlinkAdd` for addresses, `destinationAdd` + `linkAdd` for jumps within the document (0.9.4.72); the destination may come after the link.

Next chapter: Encryption

11 Encryption

11.1 Introduction

A password on the constructor switches on AES. AES-128 (`-encversion 4`, default since 0.9.4.11) and AES-256 (`-encversion 5`, since 0.9.4.16). Without a password the file stays open.

11.2 AES-128

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 \
        -userpassword "oeffnen" \
        -ownerpassword "aendern" \
        -permissions print]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Vertraulich." -x 50 -y 50
$pdf endPage
$pdf write -file geheim.pdf
$pdf destroy
```

Empty `-ownerpassword`: the user password applies to both. `-permissions` takes symbolic lists, presets or a raw `/P` number. The default acts as "everything allowed".

Password	Effect
user	opens; /P applies
owner	full rights; /P ignored

`-permissions` alone does **not** lock the file. Without a user password every viewer opens it; `/P` is a hint to conforming readers.

11.3 AES-256

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 \
        -userpassword "oeffnen" -ownerpassword "aendern" \
        -encversion 5]
```

Needs a SHA-384 backend. Pure Tcllib is slow (seconds instead of milliseconds). In production, prefer a fast crypto package.

11.4 CID Fonts: WriteObjectDirect (0.9.4.67)

Up to **0.9.4.66** the five objects of `WriteCIDFontObjects` went straight into the file, bypassing Encrypt. The document was created, was fine without a password — and unreadable **with** one:

```
qpdf: inflate: incorrect header check
pdftotext -upw … : (leer)
```

The font stream was byte-identical to the unencrypted file. A reader that obeys `/Encrypt` decrypts it anyway and gets garbage.

Since **0.9.4.67** `WriteObjectDirect` writes the same objects through Encrypt (strings and stream, key derived from the object number). Workarounds (a standard font in encrypted output) can go. Regenerate files from .66 — the content is in plain text in the old file, and the embedded font is dead for the reader.

Up to .66, XMP metadata was also written in plain text while the file claims

`/EncryptMetadata true`. `qpdf --check` does not catch this: the stream is uncompressed, so there is no inflate.

11.5 Checking

```
qpdf --password=oeffnen --check geheim.pdf
pdftotext geheim.pdf -                 # Incorrect password
pdftotext -upw oeffnen geheim.pdf -    # Text
```

All three count. The first two alone also pass for an encrypted but broken file — exactly what 0.9.4.66 produced.

11.6 Summary

- AES-128: `-encversion 4`. AES-256: `-encversion 5`.
- User password to open, owner password for full rights.
- CID plus Encrypt: readable only from 0.9.4.67 on. Rebuild old files.

Next chapter: catPdf

12 catPdf

12.1 Introduction

catPdf concatenates finished PDFs. No document object. Since 0.9.4.40 the logical structure is preserved. Since 0.9.4.44 forms are merged.

12.2 Recipe

```
package require pdf4tcl 0.9.4.69
pdf4tcl::catPdf teil1.pdf teil2.pdf gesamt.pdf
```

Options go **before** the file names (since 0.9.4.48):

```
::pdf4tcl::catPdf -title "Gesamtdatei" -author "" \
        teil1.pdf teil2.pdf gesamt.pdf
```

`-title -author -subject -keywords -creator -producer`. An empty value deletes the entry. Entries not named keep what the first document had.

Since 0.9.4.51 the values go into `/Info` **and** into XMP. Before that only `/Info` — Acrobat (which prefers XMP) kept showing the title of part 1. The packet is **edited, not rebuilt**, so that extension schemas (such as Factur-X `fx:`) are preserved.

ISO 19005-1 clause 6.7.3 requires `/Info` and XMP to match. Check against **1b**, not 2b: PDF/A-2/3 dropped the rule. The same inconsistent file: veraPDF 1b FAIL, 2b PASS.

12.3 Mixing Tagged Files

Both inputs tagged: one structure tree, with the elements of the second document attached to

the first. Page markers are kept.

Tagged plus untagged does not work cleanly. pdf4tcl writes valid PDF and appends to `::pdf4tcl::warnings`:

- tagged + untagged → later pages outside the tree
- untagged + tagged → the structure of the second document is dropped

Tag all parts the same way beforehand.

12.4 Forms (0.9.4.44)

The field list is the union. Before, only the form of the first document survived: the widgets of the others sat on the pages, but no reader offered them for filling.

Names are not made unique. Two fields with the same name are one field with a shared value (ISO 32000-1 12.7.3.2). A collision is reported in `::pdf4tcl::warnings`.

12.5 XRef Streams

Cross-reference table as a stream (PDF 1.5+): ISO 19005-1 forbids the form, PDF/A-2 and -3 require it. Since 0.9.4.49/50 catPdf reads both, object streams included.

Input	Concatenation
ordinary PDF from pdf4tcl	yes
PDF/A-1b	yes
PDF/A-2b, -3b, -3a	since 0.9.4.49
objects in containers	since 0.9.4.50

`qpdf --object-streams=generate` without `--newline-before-endstream` violates ISO 19005-2 6.1.7.1. The **input** fails in veraPDF, and so does the result — pdf4tcl passes streams through unchanged.

12.6 Limits

- Font dictionaries remain duplicated; identical font streams are shared.
- Catalog and Pages objects of appended documents remain as dead objects (a few hundred bytes).
- Orphans (unreachable `/Info` of the appended files) are dropped since 0.9.4.50.

12.7 Summary

- `catPdf ?optionen? eingabe … ausgabe` (options, inputs, output).
- Title since 0.9.4.48, XMP since 0.9.4.51.
- Structure since 0.9.4.40, forms since 0.9.4.44, XRef stream since 0.9.4.49.

Next chapter: PDF/A

13 PDF/A

13.1 Introduction

-pdfa on the constructor sets the conformance level. Without an embedded font the file fails in veraPDF — Base-14 is not enough.

13.2 Level B

```
pdf4tcl::loadBaseTrueTypeFont BaseDejaVu /pfad/DejaVuSans.ttf
pdf4tcl::createFont BaseDejaVu Body iso8859-1

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50 \
        -pdfa 2b]
$pdf metadata -title "Archivprobe" -author "pdf4tcl"
$pdf startPage
$pdf setFont 12 Body
$pdf text "Diese Datei behauptet PDF/A-2b." -x 0 -y 24
$pdf endPage
$pdf write -file archiv-2b.pdf
$pdf destroy
```

Optional: `-pdfa-icc /pfad/srgb.icc`. Otherwise pdf4tcl searches for a profile.

Level	Typical use
1b / 1a	older archive; no transparency; no embedded files
2b / 2a	modern archive; XRef stream; PDF 1.7
3b / 3a	plus embedded files (Factur-X / ZUGFeRD)

1b writes `%PDF-1.4` and a classic xref table. 2b/3b: XRef stream and `%PDF-1.7` (since 0.9.4.22).

13.3 Level U (since 0.9.4.56)

Between B and A: appearance preservable **and** text extractable as Unicode, without tagging.

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -pdfa 3u]
```

No 1u: the level came with part 2. CID fonts carry ToUnicode; whatever passes B with an embedded font passes U. Use: an archive without a structure tree, typically scan plus OCR.

13.4 Level A (since 0.9.4.41)

A requires tagged PDF and a document language. Both are checked at `finish`; if one is missing, no file with a false `pdfaid:conformance A` is produced:

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -pdfa 3a]
$pdf tagged 1 -lang de-DE -ua 1
$pdf metadata -title "Zugängliches Archiv"
$pdf viewerPreferences -displaydoctitle 1
$pdf startPage
$pdf setFont 12 Body
$pdf tagText H1 "Titel" -x 0 -y 24
$pdf tagBegin P
$pdf text "Getaggter Fließtext." -x 0 -y 50
$pdf tagEnd
```

A does not check whether the tagging makes sense — only that it is there. Every paragraph a `/P` and every heading an `/H1` validates and tells a reader nothing useful.

13.5 Pitfalls

- Standard font under `-pdfa`: warning since 0.9.4.41, the file fails in veraPDF.
- `setAlpha` below 1 with PDF/A-1: warning since 0.9.4.22.
- Embedded files with PDF/A-1: error. Factur-X: 3b/3a.
- Check marks: vector under conformance, see Creating Forms.

```
verapdf -f 2b archiv-2b.pdf
verapdf -f 3a archiv-3a.pdf
verapdf -f ua1 archiv-3a.pdf
```

13.6 Summary

- `-pdfa 1b|2b|3b|1a|2a|3a|2u|3u`, always with an embedded font.
- A: `tagged 1 -lang …` before `finish`.
- U: Unicode without a structure tree.

Next chapter: Layers

14 Layers

14.1 Introduction

The reader switches optional content (OCG, optional content groups) on and off in the viewer: debug grid, letterhead, watermark. Layers apply to the whole document; `beginLayer` / `endLayer` enclose the drawing on a page.

14.2 Recipe

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1]
set gitter [$pdf addLayer "Debuggitter" -visible 0]
set rumpf [$pdf addLayer "Rumpf" -visible 1]
```

```
$pdf startPage

$pdf beginLayer $gitter
$pdf setStrokeColor 0.85 0.85 0.85
for {set x 0} {$x <= 500} {incr x 50} {
    $pdf line $x 0 $x 700
}
$pdf endLayer

$pdf beginLayer $rumpf
$pdf setFont 12 Helvetica
$pdf text "Sichtbarer Rumpf" -x 50 -y 50
$pdf endLayer

$pdf endPage
$pdf write -file ebenen.pdf
$pdf destroy
```

`addLayer` returns an ID for `beginLayer` / `endLayer` and for `-layer` on annotations and forms.

14.3 Options (since 0.9.4.63)

```
$pdf addLayer "Vordruck" -print 0
$pdf addLayer "Detail"   -zoom {2.0 {}}
$pdf addLayer "Kopf DE"  -group kopf
set liste [$pdf layers]
```

Option	Effect
-visible	initial state (default 1)
-print 0	on screen, not on paper
-export	state when saving to a format without OCG
-zoom {min max}	on only at this magnification; empty = unlimited
-locked	the viewer must not toggle it
-group name	at most one layer of the group visible

`-print 0` writes `/Usage << /Print << /PrintState /OFF >> >>`. ISO 32000-1 8.11.4.4: usage dictionaries apply to **interactive** conforming readers, not to final-form output. Printing from the viewer honors it; a RIP does not, and that is conforming. Where it must not go wrong, do not put the graphics into the file at all.

As soon as a layer carries `/Usage`, the default configuration gets an `/AS` array — without it no viewer applies `/Usage`.

14.4 Counting

`endLayer` without `beginLayer` is an error (formerly a stray `EMC`). `endPage` rejects a page with an open layer (ISO 32000-1 14.6).

14.5 Forms

Annotations lie outside the content stream. `beginLayer` does not reach them. `-layer` on `addForm` writes `/OC` into the annotation dictionary.

14.6 Summary

- `addLayer`, then `beginLayer` / `endLayer` on the page.
- `-print 0` and `-zoom` via `/Usage`; `/AS` is created automatically.
- An open layer at the end of a page is an error.

Next chapter: Metadata

15 Metadata

15.1 Introduction

Title, author and subject are stored twice: in the classic `/Info` and in the XMP packet the catalog points to. `metadata` writes both. Acrobat prefers to read XMP.

15.2 Setting

```
$pdf metadata \
        -title "Jahresbericht 2026" \
        -author "Muster GmbH" \
        -subject "Kennzahlen" \
        -keywords "Bericht, 2026" \
        -creator "bericht.tcl" \
        -producer "pdf4tcl"
```

Unknown option:

```
unknown metadata option "-foo"
```

Values that are empty after `string trim` are ignored. Dates:

```
$pdf metadata -creationdate 0 -moddate 0
```

`0` means "now" (`clock seconds`). Otherwise a PDF date that `_ValidatePdfDate` accepts.

15.3 Viewer

```
$pdf viewerPreferences -displaydoctitle 1 -pagemode UseOutlines
```

`-displaydoctitle 1` shows the document title in the title bar instead of the file name. PDF/A

level A and PDF/UA often expect this.

Further switches: `-pagelayout`, `-hidetoolbar`, `-fitwindow`, `-centerwindow`, `-direction L2R|R2L`.

15.4 Attachments

```
$pdf addEmbeddedFile "notizen.txt" \
        -contents "Beilage\n" \
        -mimetype "text/plain" \
        -description "Nebennotizen" \
        -afrelationship Data
```

`-afrelationship`: `Alternative`, `Data`, `Source`, `Supplement`, `Unspecified`. Forbidden under `-pdfa 1b` (error). PDF/A-3b enters the file into `/AF` automatically. Factur-X: use the `facturx` method, do not build the packet by hand.

15.5 Encryption

Since 0.9.4.67 the XMP stream also goes through Encrypt when the file sets `/EncryptMetadata true`. Up to 0.9.4.66 the title was in plain text in a supposedly encrypted file. `qpdf --check` does not see this.

15.6 catPdf

Concatenation takes over the catalog of the first part. Set the title of the combined file with `catPdf -title …`; otherwise the title of part 1 remains — in XMP too, since 0.9.4.51. See catPdf.

15.7 Summary

- `metadata` fills `/Info` and XMP.
- `viewerPreferences -displaydoctitle 1` for archive and UA.
- Attachments: PDF/A-3, not PDF/A-1.

Next chapter: gsave and Text

16 gsave and Text

16.1 Introduction

`text` leaves the text object open so that several calls share one `BT`. `q` (`gsave`) and `Q` (`grestore`) are graphics operators. ISO 32000-1 section 8.2 allows only text operators between `BT` and `ET`.

16.2 The Trap Up to 0.9.4.61

`gsave` and `grestore` wrote `q` / `Q` straight into the open text object:

```
BT … q … ET Q
```

Poppler, MuPDF and pdftotext rendered the invalid stream anyway. **Adobe Reader showed a blank page without any message.** `pdfinfo` reported a healthy file.

Found through `pdf4tcllib::labels::render` with `-clip 1`. `tests/nesting.test` checks the class of error, not a string.

16.3 Since 0.9.4.62

Both methods call `EndTextObj` **before** `q` / `Q`. Nothing changes at the call. Anyone comparing generated files byte by byte sees `q` and `Q` outside `BT`/`ET`, and an `ET` may appear where there was none.

```
$pdf setFont 12 Helvetica
$pdf text "Zeile eins" -x 50 -y 50
$pdf gsave
$pdf clip 40 40 200 80
$pdf rectangle 40 40 200 80 -filled 1
$pdf grestore
$pdf text "Zeile zwei" -x 50 -y 80
```

After `gsave` the next `text` starts a new text object. That is correct, not a regression.

16.4 Other Methods

`clip`, `rectangle` and the line path close the text object themselves or through `DrawLine`. Only `gsave` / `grestore` did not — until 0.9.4.62.

Transformations (`translate`, `rotate`, `scale`) belong between `gsave` and `grestore`. Text set with `Tm` does not follow `cm`; see Graphics. Still, save the graphics state first; otherwise the matrix stays skewed for everything that follows.

16.5 Rule

After `text` and before any graphics operation that writes `q`, `cm`, `W` or path operators: either use a drawing method that calls `EndTextObj` itself, or call `gsave` explicitly. Since 0.9.4.62 `gsave` is enough.

Keep Adobe as a test reader. A page that is full in Poppler and blank in Acrobat is often exactly this stream.

16.6 Tagged Text Since 0.9.4.69

`TagEnsureMC` writes `BDC`/`EMC` outside `BT`/`ET` and may close an open text object to do so. If the call came between `Tm` and `Tj`, `Tj` ended up outside:

```
BT / Tm / ET / << /MCID n >> BDC / (…) Tj / EMC
```

ISO 32000-1 9.4: text output belongs between `BT` and `ET`. Adobe does not draw it outside; Poppler, MuPDF and Chrome do. Since 0.9.4.69 the marking comes **before** `BeginTextObj`. If

TagEnsureMC closes an open text object, Tm is written again (posSet).

16.7 Summary

- text keeps BT open.
- gsave / grestore close BT before q / Q (0.9.4.62).
- Tagged: marking before BT, otherwise Tj ends up outside (0.9.4.69).
- Adobe: blank page without a message when q lands inside BT or Tj outside BT/ET.

Next chapter: CID Subset and CMap

17 CID Subset and CMap

17.1 Introduction

Since 0.9.4.57 pdf4tcl writes only the glyphs used for CID (TrueType). The subset must be a valid TrueType file: required tables, cmap, checksums. Up to **0.9.4.67** the cmap and real table checksums were missing. **0.9.4.68** adds both.

Callers change nothing. createFontSpecCID and setFont stay the same. This chapter explains why Adobe showed a blank page and why letzteGlyphe (last glyph) is numGlyphs-1, not 0xFFFE.

17.2 cmap Is Required

cmap is listed among the required tables (Apple TrueType, OpenType). Up to 0.9.4.67 CIDRebuildTtf removed it: Identity-H addresses glyphs directly. That holds for the PDF page — and that is why nobody noticed:

- Viewers with /CIDToGIDMap /Identity drew the glyphs
- pdftotext extracted the right text
- qpdf found no syntax error
- **Adobe Reader showed a blank page**

The subset simply was not a valid font. Whether Reader really draws from this table has not been checked. The PDF standard says that for an embedded CIDFontType2 the CIDToGIDMap applies. The likely cause: Reader validates the font program. The fix rests on the cmap being required, not on the assumption.

17.3 Identity, Not Unicode

A viewer loop through the cmap would read the glyph ID as a character code. Glyph 0x5A is not U+005A. Both paths must find the same glyph. That is why CIDIdentityCmap writes an identity mapping, not the original Unicode cmap.

Two segments, format 4:

- 0.. letzteGlyphe maps onto itself
- 0xFFFF is the mandatory sentinel and points to glyph 0

17.4 letzteGlyphe Is numGlyphs-1

A cmap entry must not point to a glyph that does not exist. An earlier draft ran the first segment up to `0xFFFE`. With DejaVuSans, 59282 codes then pointed into nothing; fontTools rejected the font. `cidfont-11.2` covers both ends.

`CIDIdentityCmap` clamps: below 0 becomes 0, above `0xFFFE` becomes `0xFFFE`. The call from `CIDRebuildTtf`:

```
CIDIdentityCmap [expr {$numGlyphs - 1}]
```

`numGlyphs` is in `maxp` at offset 4.

17.5 Checksums (0.9.4.68)

Two places:

checkSumAdjustment in `head` is set to 0 before the table checksum is computed. The original value belongs to the **original font** and is wrong for a subset. The standard computes the table checksum of `head` with this field set to 0. The final value comes at the very end, once the file is complete.

Checksum per table. Up to 0.9.4.67 this was 0. The generated file was not a valid TrueType file — so clearly that `loadBaseTrueTypeFont` rejected it with `invalid TTF file checksum`. `MakeTTFSubset` (the other subsetter) has always done this correctly; only the CID path did not.

Layout tables (GPOS, GSUB, GDEF, kern, MATH, FFTM, DSIG) are left out: they are not required, and pdf4tcl already has their content in `BFA` from loading.

17.6 CFF

OpenType/CFF is not subset. There is a warning, and the whole face goes into the PDF. The same cmap fix does not apply there.

17.7 Summary

- Up to 0.9.4.67: no cmap in the CID subset; Adobe blank, other viewers full.
- 0.9.4.68: `CIDIdentityCmap` with `numGlyphs-1`, not `FFFE`.
- Table checksums and `checkSumAdjustment` 0, then the final value.
- API unchanged.

Next chapter: Case Study: Report

18 Case Study: Report

18.1 Introduction

A one-page report: CID font, header, three key figures, a form field, a stamp. The parts come from the previous chapters, here combined in one sequence.

18.2 Font and Document

```
package require pdf4tcl 0.9.4.69

pdf4tcl::loadBaseTrueTypeFont BaseDejaVu \
        /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
pdf4tcl::createFontSpecCID BaseDejaVu Bericht

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 48 \
        -userpassword "lesen" -ownerpassword "setzen" \
        -permissions {print}]
$pdf metadata -title "Monatsbericht März 2026" -author "Muster GmbH"
$pdf startPage
$pdf setFont 16 Bericht
```

Encryption plus CID: readable since 0.9.4.67. The user may print, not copy.

18.3 Header and Key Figures

```
$pdf text "Monatsbericht März 2026" -x 0 -y 8
$pdf setFont 10 Bericht
$pdf text "Muster GmbH — intern" -x 0 -y 28

$pdf setStrokeColor 0.2 0.2 0.2
$pdf setLineWidth 0.5
$pdf line 0 40 500 40

set kasten {{Umsatz 128} {Aufträge 41} {Offen 7}}
set x 0
foreach paar $kasten {
    lassign $paar label wert
    $pdf setFillColor 0.93 0.93 0.93
    $pdf rectangle $x 56 150 48 -filled 1
    $pdf setFillColor 0 0 0
    $pdf text $label -x [expr {$x+10}] -y 64
    $pdf setFont 18 Bericht
    $pdf text $wert -x [expr {$x+10}] -y 84
    $pdf setFont 10 Bericht
    incr x 170
}
```

After a gray fill, reset the fill color to black; otherwise the text inherits the gray.

18.4 Field and Stamp

```
$pdf text "Freigabe:" -x 0 -y 130
$pdf addForm text 80 120 200 16 -id freigabe -maxlen 24

$pdf addAnnotStamp 400 500 90 28 -name Approved -color {0.2 0.5 0.2}
```

`setFont` was already set. Without a font the stamp would stay empty in PDFium (Annotations).

```
$pdf endPage
$pdf write -file bericht.pdf
$pdf destroy
```

18.5 Filling Later

`fillForms` and `getForms` do not read encrypted files (no `-password`). On the AES-encrypted `bericht.pdf` from above, `fillForms` fails with `no such field(s) … freigabe`. For the fill step, write the same drawing **without** `-userpassword`/`-ownerpassword`:

```
# Konstruktor ohne Kennwörter, sonst wie oben …
$pdf write -file bericht-offen.pdf
$pdf destroy
pdf4tcl::fillForms bericht-offen.pdf bericht-frei.pdf \
        {freigabe "Müller, 28.03."}
```

Simple text field: the appearance is rebuilt (0.9.4.64). NeedAppearances stays off, so the border of an untouched check box — there is none here — would not be lost.

Check:

```
qpdf --check bericht-frei.pdf
pdftotext bericht-frei.pdf -
```

18.6 What Is Deliberately Missing

- Page breaks and tables: `pdf4tcllib`
- PDF/A: drop Base-14, `-pdfa 2b`, `metadata` stays
- A second page with an attachment via `catPdf`
- Comb field without MaxLen: do not fill it and expect an appearance

This is the core that the how-tos take apart. Here it stands as one application.

18.7 Summary

- Load CID, document with AES, draw the header, field, stamp, write.
- fillForms for the text field; stamp only after setFont.
- Layout across pages: not in this package.

Next chapter: Pitfalls

19 Pitfalls

A short list. The reasons are in the chapters.

19.1 Text and Fonts

- Helvetica **-Oblique**, Times **-Italic**.
- Base-14: everything outside Latin-1 becomes?. Under 8.6 `getSubstCount` is often 0 despite question marks.
- Encoded subset: put? (63) into the list, otherwise the fallback is slot 0.
- `drawTextBox -newyvar`: caller coordinates since 0.9.4.44; remove the old `pageHeight` correction.
- Soft hyphen since 0.9.4.61 without a glyph and without width.
- CFF/OpenType: CID does not subset, warning, whole face.

19.2 Graphics

- Color 0..1, not 0..255 (error since 0.9.4.39).
- `setBgColor` is not the page background.
- `q` in an open `BT`: Adobe blank without a message. Since 0.9.4.62 `gsave` closes the text object.
- Tagged before 0.9.4.69: `Tj` outside `BT`/`ET`, Adobe blank, Poppler full. Marking before `BeginTextObj`.
- Text does not follow `cm`.

19.3 Canvas

- `$pdf canvas: update`, then `-bbox`.
- tko matrix flat `{a b c d tx ty}`, tkpath nested.
- `-fontmap`: canvas the whole specification, tko only the family.
- window items often missing.

19.4 Forms and Annotations

- fillForms builds the appearance for a simple text field (0.9.4.64).
- Comb without MaxLen, password, choice: old stream.
- NeedAppearances only there, otherwise PDFium wipes untouched check boxes.
- Field name `person.city`, not `city`.
- Stamp after `setFont`, otherwise PDFium shows nothing.
- Dynamic XFA: fillForms refuses.

19.5 Output

- CID plus Encrypt: rebuild files from before 0.9.4.67.
- CID subset before 0.9.4.68: Adobe blank (no cmap / checksum 0).
- PDF/A: embed the font; Base-14 gives only a warning, not a valid archive.
- catPdf: set the title, otherwise the title of part 1; XMP since 0.9.4.51.
- Open layer at `endPage`: error.

Next chapter: API

20 API

Quick reference for pdf4tcl 0.9.4.69, links within the document from 0.9.4.72. Options are incomplete; the manpage remains authoritative.

20.1 Package and Object

```
package require pdf4tcl 0.9.4.69
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf endPage
$pdf write -file out.pdf
$pdf destroy
```

Constructor (selection): `-paper -orient -landscape -margin -unit -compress -cmyk -pdfa -pdfa-icc -markstyle -userpassword -ownerpassword -encversion -permissions`.

`write -file` / `-chan`. Warnings: `::pdf4tcl::warnings`.

20.2 Text and Fonts

```
$pdf setFont 12 Helvetica
$pdf text "Zeile" -x 0 -y 24
$pdf drawTextBox x y w h $txt -align left -newyvar nextY
$pdf getStringWidth $txt
$pdf getStringWidth $txt -font Helvetica -size 12
$pdf getSubstCount

pdf4tcl::loadBaseTrueTypeFont Base datei.ttf
pdf4tcl::createFont Base Body iso8859-1
pdf4tcl::createFontSpecEnc Base Body $codepoints
pdf4tcl::createFontSpecCID Base Uni
```

20.3 Graphics

```
$pdf setFillColor red
$pdf setStrokeColor 0 0 0
$pdf setLineWidth 1
$pdf setAlpha 0.5
$pdf line x1 y1 x2 y2
$pdf rectangle x y w h -filled 1
$pdf circle x y r
$pdf gsave
$pdf clip x y w h
$pdf linearGradient x1 y1 x2 y2 c1 c2
$pdf translate dx dy
$pdf rotate winkel
$pdf scale sx sy
$pdf grestore
```

20.4 Images and Canvas

```
set id [$pdf addImage datei.png -id logo]
$pdf putImage $id x y -width 120
$pdf canvas .c -bbox $bb -x 50 -y 50 -width 400 -height 300 \
        -sticky nw -fontmap {DocText Uni}
```

20.5 Form, Annotation, Layer

```
$pdf addForm text x y w h -id name -init "" -maxlen 8 -comb 1
pdf4tcl::getForms in.pdf
pdf4tcl::fillForms in.pdf out.pdf {name Wert}
pdf4tcl::exportForms in.pdf out.fdf

$pdf addAnnotStamp x y w h -name Approved
$pdf addAnnotNote x y w h -content "…"
$pdf addAnnotLine x1 y1 x2 y2
$pdf hyperlinkAdd x y w h url                   ;# Adresse
$pdf destinationAdd name ?-y y? ?-x x?          ;# Ziel (0.9.4.72)
$pdf linkAdd x y w h name                       ;# Sprung (0.9.4.72)

set l [$pdf addLayer "Gitter" -visible 0 -print 0]
$pdf beginLayer $l
$pdf endLayer
```

20.6 Concatenation, Metadata, PDF/A

```
pdf4tcl::catPdf -title "Gesamt" a.pdf b.pdf out.pdf
$pdf metadata -title "…" -author "…"
$pdf viewerPreferences -displaydoctitle 1
$pdf tagged 1 -lang de-DE -ua 1
$pdf addEmbeddedFile name.txt -contents $bytes -mimetype text/plain
```

Constructor `-pdfa 1b|2b|3b|1a|2a|3a|2u|3u`. `-encversion` 4 AES-128, 5 AES-256.

Next chapter: Glossary

21 Glossary

21.1 A

AES-128 / AES-256
: Encryption via `-encversion` 4 or 5. The user password opens, the owner password grants full rights.

Appearance stream
: The drawing of a field or an annotation. Without it PDFium does not show stamps. fillForms

rebuilds it for a simple text field.

21.2 B

Base-14
Fourteen standard fonts, not embeddable. WinAnsi only.

21.3 C

catPdf
Concatenates PDFs. Structure since 0.9.4.40, forms since 0.9.4.44.

CID font
Type 0 with Identity-H, Unicode via TrueType. Subset since 0.9.4.57.

CIDIdentityCmap
Identity cmap of the subset since 0.9.4.68. `letzteGlyphe` (last glyph) = `numGlyphs-1`.

cmap
Required TrueType table mapping characters to glyphs. Removed from the CID subset up to 0.9.4.67.

21.4 N

NeedAppearances
Flag on the AcroForm: the viewer should rebuild appearances. fillForms sets it only where the stream stays old.

21.5 O

OCG
Optional content group, layer. `addLayer` / `beginLayer` / `endLayer`.

orient
`-orient 1` (default): Y from the top.

21.6 P

PDF/A
Archival format. Embed fonts. Levels B, U, A; parts 1–3.

PDFium
The viewer in Chrome and Edge. Stamps and lines without an appearance are invisible. NeedAppearances empties Off check boxes.

21.7 T

Text object
BT … ET. `text` keeps it open. Since 0.9.4.62 `gsave` closes it before `q`.

ToUnicode
Mapping from glyph to Unicode for extraction. Base-14 since 0.9.4.9 for WinAnsi; CID always.

21.8 W

WriteObjectDirect
Writes an object through Encrypt. Required for CID font objects since 0.9.4.67.

21.9 X

XMP

Metadata packet on the catalog. `metadata` and `catPdf -title` write `/Info` and XMP (catPdf since 0.9.4.51).

22 Error Messages

Common messages and their usual cause.

22.1 no font set

`addForm` for `combobox`, `listbox`, `pushbutton` or `signature` before `setFont`. Text fields and check boxes may come earlier. Since 0.9.4.63; formerly `can't read "FontsAttrs(,specialencoding)"`.

22.2 unknown color: "1" -- expected a name like "red", "#rrggbb", …

The color is not a 0..1 triple, not hex, not a table name. `1` alone is invalid. Since 0.9.4.63 the message includes an example.

22.3 unknown color / invalid … component

A component outside 0.0.. 1.0 (since 0.9.4.39). Replace `255 0 0` with `1 0 0` or `#ff0000`.

22.4 createFontSpecCID: base font '…' not loaded

`loadBaseTrueTypeFont` is missing or the base name differs.

22.5 invalid TTF file checksum

A damaged or incomplete TrueType file. CID subsets before 0.9.4.68 carried a table checksum of 0 and failed here when loaded again.

22.6 fillForms: no such field(s)

The name is not in the form. Since 0.9.4.64 the full chain: `person.city`, not `city`. Do not ignore typos — otherwise the form stays empty without explanation.

22.7 fillForms: "…" is not an option of "…"

The choice value is neither an export value nor a label. An empty value is allowed. Since 0.9.4.65.

22.8 unknown form type …

The type is not in `text checkbutton combobox listbox password radiobutton pushbutton signature`. Alias: `checkbox` for `checkbutton`.

22.9 unknown metadata option "…"

Only `-author -creator -keywords -producer -subject -title -creationdate -moddate`.

22.10 papersize "..." is unknown

Neither an ISO name nor a two-element list `{breite höhe}` (width, height).

22.11 rotation ... not a multiple of 90

`-rotate` accepts only multiples of 90.

22.12 endLayer without beginLayer / layer still open

Layer calls are not paired. endPage with an open layer is an error (0.9.4.63, ISO 32000-1 14.6).

22.13 Adobe: Blank Page, No Message

No throw. q inside BT (before 0.9.4.62), a CID subset without a cmap (before 0.9.4.68) or tagged `Tj` outside BT/ET (before 0.9.4.69). Often full in Poppler. See gsave and Text and CID Subset and CMap.

22.14 qpdf: inflate: incorrect header check

An encrypted PDF with CID from 0.9.4.66 or earlier. Rebuild with 0.9.4.67+. See Encryption.

22.15 PDF/A: the standard font Helvetica has no embeddable font program

A warning in `::pdf4tcl::warnings`, no abort. veraPDF fails. Load a TrueType font.

22.16 pdf4tcl: linkAdd target(s) never defined with destinationAdd: ...

A warning on stderr when writing, no abort. A `linkAdd` names a name that no `destinationAdd` created; the link has no effect. Make the names match. See Annotations.

22.17 destination "..." already defined

The same name passed twice to `destinationAdd`. Names are unique per document.

22.18 invalid destination name "...": must match [A-Za-z0-9._-]+

A space, umlaut or special character in the name of a destination or link.

End of the appendices. Start: Understanding pdf4tcl

23 License


```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)
The author's code examples: MIT License (below)

---
```

Text

This book is licensed under the Creative Commons license "Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0).

You may copy and redistribute the material in any medium or format and remix, transform and build upon it, for any purpose, even commercially, under the following terms:

* Attribution: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

* ShareAlike: If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

The full license text is binding:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Code examples

The author's code examples (in the code blocks of the book and as scripts next to it) are additionally released under the MIT License. Code examples and quoted text taken from other projects (Tcl/Tk, pdf4tcl and others) keep the license of their original; the source is named where they appear.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

```
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Index

-

-pdfa 27

A

AES 23
appearance stream 17

C

canvas 15
catPdf 25
CID font 6, 10
CIDIdentityCmap 33

D

destinationAdd 22
document 7

F

fillForms 19

H

hyperlinkAdd 22

L

linkAdd 22

M

matrix 16

N

NeedAppearances 20

O

OCG 28
optional content group 8

P

pdf4tcl 6

S

standard fonts 9

T

text object 9, 31